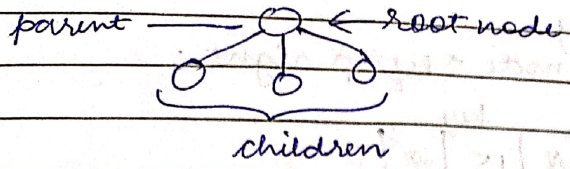
12/10/22

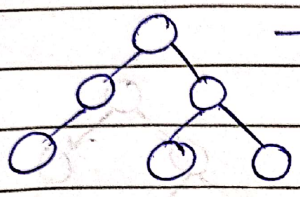
Page No.

TREE



Binary tree

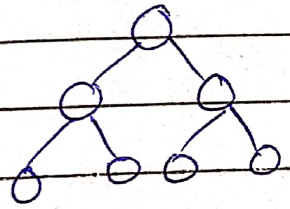
0, 1 or 2 children permitted



	(max # nodes)	
level 0	1	level i having max 2^i nodes.
level 1	2	
level 2	4	

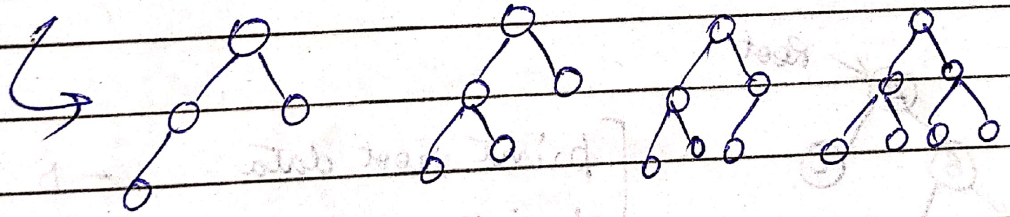
Height of tree / depth of tree = # of levels $h=3$

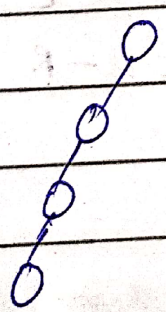
$n = 2^h - 1$ $\quad h = \lceil \log_2(n+1) \rceil$



each level is full (Full tree)

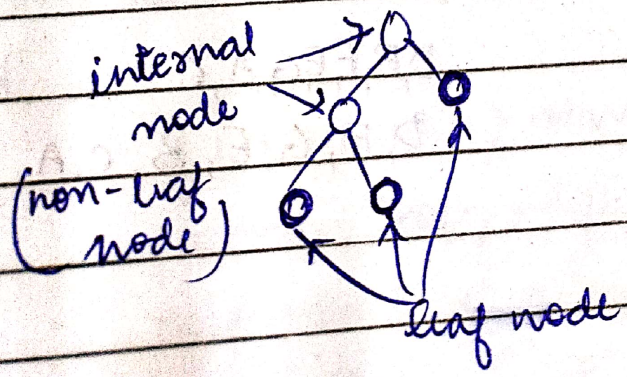
Complete tree → All levels except last one are full.



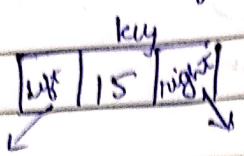


Skewed tree

$n=4$ $h=4$



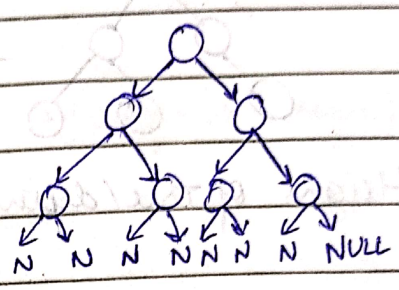
```
struct node {
    int key;
    struct node *left, *right;
};
```




```
struct node *root;
root = NULL;
```

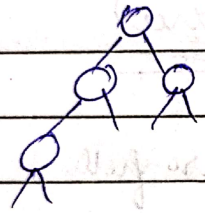
No. of non-NULL pointers = n-1

n-nodes → 2n pointers.



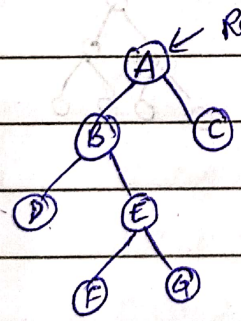
⟹ 2n-(n-1) ptrs. are NULL

n+1



3 Non-NULL

5 NULL



print root data — D
print left subtree — L
print right subtree — R

A, B, D, E, F, G, C

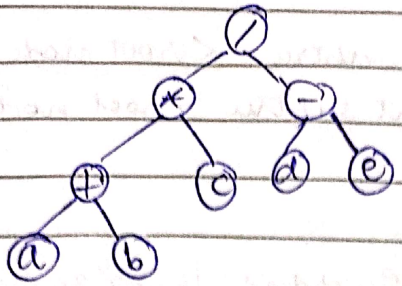
DLR → preorder

LDR → inorder — D, B, F, E, G, A, C

LRD → post order — D, F, G, E, B, C, A

Page No.

(a+b) * c / (d-e)



preorder: /*+abc-de (prefix)

postorder: ab+c*de-/ (postfix)

13/10/22

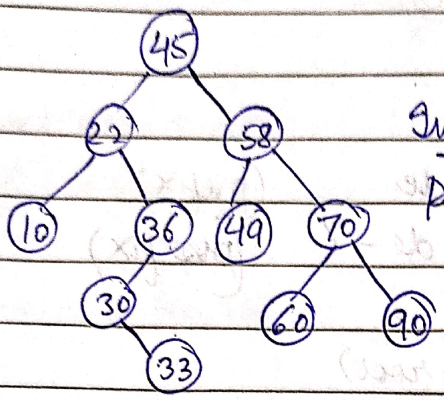
```
void inorder (struct node *root)
{
    if (root==NULL) return;
    inorder (root->left);
    printf ("%d\t", root->key);
    inorder (root->right);
}

void preorder (struct node *root)
{
    if (root==NULL) return;
    printf ("%d\t", root->key);
    preorder (root->left);
    preorder (root->right);
}

Void postorder (struct node *root)
{
    if (root == NULL) return;
    postorder (root->left);
    postorder (root->right);
    printf ("%d\t", root->key);
}
```

Binary Search Tree (BST)

data(keys) stored in left subtree < root node key
data (keys) stored in right subtree > root node key



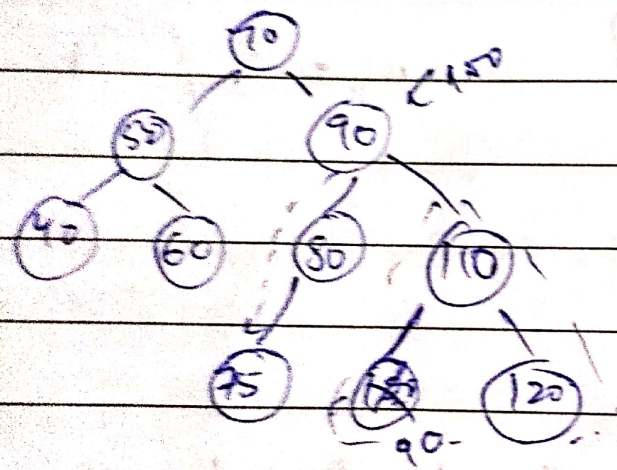
Inorder: 10, 22, 30, 33, 36, 45, 49, 58, 60, 70,
preorder: 45, 22, 10, 36, 30, 33, 58, 49, 70,

```
boolean search(struct node * root, int x)
{
    if (root == NULL) return (false);
    if (x < root -> key)
        return (search(root -> left, x));
    else if ( x > root -> key)
        return (search(root -> right, x));
    else return(true);
}
```

```
Void Insert (struct node ** r, int x)
{
    struct node *p, *q;
    p = getnewnode();
    p -> key = x;
    p -> left = NULL;
    p -> right = NULL;
    if (*r == NULL)      // empty tree
        *r = p;
```

Date : / /
Page No.

```
else
{   q=*r;
    while(1)
    {
      if (x<q->key && q->left != NULL)
            q= q->left;
      else if (x>q->key && q->right != NULL)
            q=q->right ;
      else if break;
    }
    if (x<q->key)
          q->left=p;
    else q->right=p;
  } //end else
}
```



delete 40

case 1: It is leaf

delete 80

case 2: It has one child only

delete 90

case 3: It has both children

```
struct treenode delete (struct treenode *root, int x)
{
    if (root == NULL) return root;
    if (root->key > x)
        root->left = delete (root->left, x);
        return root;
    else if (root->key < x){
        root->right = delete (root->right, x);
        return root;
    }
```

if (root → left == NULL && root → right == NULL) { free(root); return NULL; }

if (root → left == NULL) {

temp = root → right; free(root); return temp;

}

if (root → right == NULL) { temp = root → left; free(root); return temp; }

// both left and right children are there

y = getsmallest (root → right);

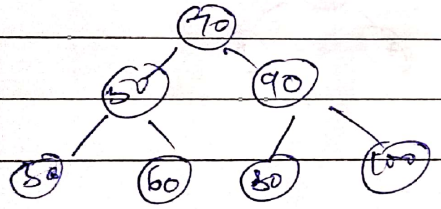
root → key = y;

root → right = delete (root → right, y);

return root;

}

Level order traversal



Queue [70 | | | |]

while (queue is not empty)

{

p = delete ();

print (p → key);

if (root → left != NULL) insert (root → left);

if (p → right != NULL) insert (p → right);

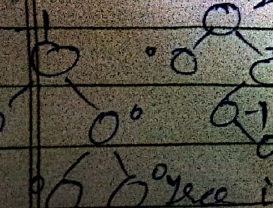
}

height()
countnodes()
smallest()
largest()
copy()
createmirror()
isBST()

Worst case complexity of search in tree → O(N) [The tree may be skewed]

AVL trees

Height balanced trees

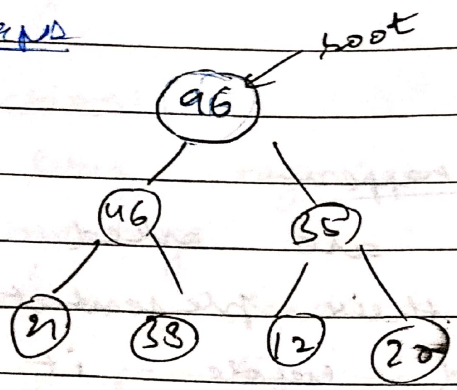


Balance factor for each node

b.f = hl − hr

Tree is balanced if all nodes bf = 0, −1, +1

Heaps



value stored in parent node is greater than values in left and right child.

96	46	35	21	38	12	20	x
0	1	2	3	4	5	6	7

p^{2i+1}

Parent node at index = i

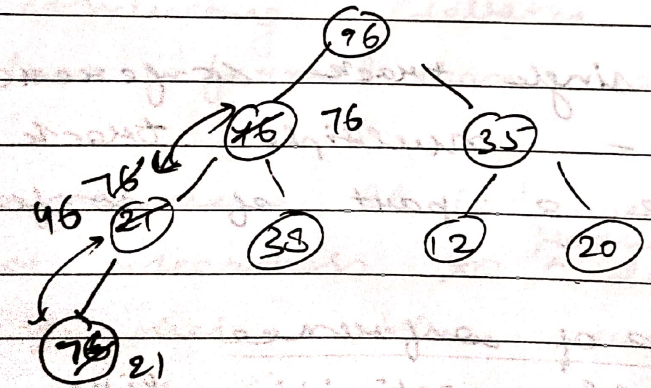
Left child of node = $2i+1$

Right child of node = $2i+2$

For a node at index i,

its parent node index = $\left\lfloor \frac{i-1}{2} \right\rfloor$

Inserting a node (x = 76)



compare and swap if required

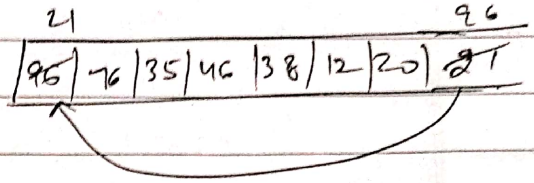
```
Insert (A, n, x)
// Insert x into Heap [0 .. n-1]
{
    A[n] ← x;
    n ← n+1;
    i ← n-1;
    p ← (i-1)/2;
```

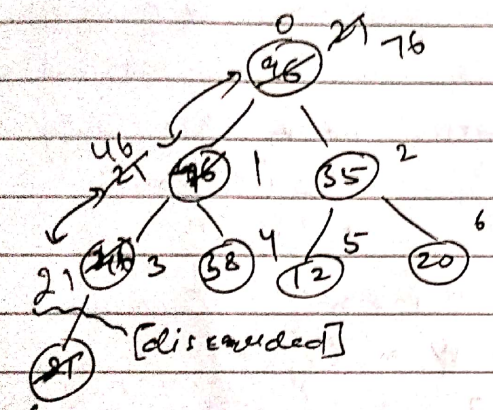
Page No.
Date:

```
while (p >= 0 && A[i] > A[p])
{
    swap (A[i], A[p]);
    i ← p;
    p ← (i-1)/2;
}
```

3
3

Delete node






```
deleteMax (A, n)
{
    x ← A[0];
    A[0] ← A[n-1];
    n--;
    i ← 0;
    while (true)
    {
        left ← 2*i+1;
        right ← 2*i+2;
        largest ← i;
        if (left < n && A[left] > A[largest])
            largest ← left;
        if (right < n && A[right] > A[largest])
            largest ← right;
        if (largest != i)
        {
            swap (A, i, largest);
            i ← largest;
        }
        else break;
    }
    return (x);
}
```